

translations

Translations — Mandatory HelixTranslate Standard (§11.4.133)

This document summarizes the inherited translation mandate, the canonical pipeline, and the validation workflow in human-readable form. The authoritative source is **Constitution.md §11.4.133**; this is a digest, not a substitute.

The mandate (in one sentence)

Every project that inherits this constitution **MUST** generate **ANY** translation it needs through **HelixTranslate** (`git@github.com:HelixDevelopment/HelixTranslate.git`) via the canonical universal pipeline in `scripts/translation/` . Hard rule — no exceptions.

What it covers

ALL content rendered in a non-source language: UI strings, documents, articles, CVs, READMEs, marketing copy, in-app messages, release notes, legal text — any human-language string. There is no category of content exempt from this mandate.

The four hard rules

1. **No hand-translation.** No human-authored or manually edited translation, ever. The engine is the single source of every translated string.
2. **No other engines.** No Google Translate, DeepL, raw ChatGPT/Claude prompting, OS translation APIs, or library i18n auto-translators. Exactly one engine.

3. **No silent fallback.** No quiet source-string substitution, no empty/partial output passed off as a translation, no swallowed error. On exhaustion of all providers the pipeline FAILS LOUD — non-zero exit, destination not written. (Provider failover *within* HelixTranslate, primary → fallback with logged retries, is allowed.)
4. **Mandatory validation.** Every translation is validated for accuracy with real, captured evidence. Unvalidated/unevidenced translation = ABSENT.

The canonical pipeline

Location: `scripts/translation/`

- `translate-pipeline.sh` — universal Markdown / plain-text translator.
 - **plain** mode: translates the whole file.
 - `--article` mode: keeps YAML frontmatter byte-identical (slugs, repos, tech, title, teaser are proper nouns / URLs / language-neutral and must survive so links and rendering keep working); translates ONLY the body, then reassembles.
- `render-articles.sh` — renders translated article sources to standalone HTML fragments for embedding (e.g. "Read more" modals).

```
# Translate:
translate-pipeline.sh --in src.md --out dst.ru.md --lang ru          #
                        plain
translate-pipeline.sh --in src.md --out dst.sr.md --lang sr --article #
                        article

# Render to HTML fragments:
render-articles.sh <site-root> ru sr
```

Language → script: `ru` → cyrillic, `sr` → latin.

Provider routing (explicit, recorded)

Role	Provider	Model
Primary	groq	llama-3.3-70b-versatile
Fallback	mistral	mistral-large-latest

Failover: retry the primary up to 3× with linear backoff, then fail over to the fallback. The provider that actually produced each output is recorded in the per-file log. Routing is never implicit — any deviation must be stated in configuration and logged.

Engine binary: supplied by the consuming project via `HELIX_TRANSLATE_BIN`, built from the HelixTranslate repository.

Validation workflow

1. **Run** the pipeline for each source → target pair. It calls the HelixTranslate engine with explicit `provider / model / source-lang / target-lang / script`.
2. **Evidence is captured automatically** to a per-file log under `$TRANSLATE_EVIDENCE_DIR` (default `<project>/_tests/evidence/translate/`): the provider used, byte counts, each retry, and success/failure are appended per run.
3. **Fail-loud check**. If all providers are exhausted, the script exits non-zero and does NOT write the destination — callers MUST treat this as a hard error, never ship the source string in its place.
4. **Accuracy validation**. Validate each produced translation for accuracy with real (physical) evidence (per §11.4.5 / §11.4.69 / §11.4.107). The run log is the evidence artifact. An unvalidated or unevidenced output is treated as ABSENT.
5. **Re-runnable / idempotent**. Each invocation recomputes the output from source and overwrites the destination, so re-validation always reflects current source.

Inheritance

This is the inherited standard. Consuming projects restate + cite §11.4.133 via §11.4.35 inheritance. Non-compliance is a release blocker; there is no escape hatch.

See: `scripts/translation/README.md` and `Constitution.md` §11.4.133.

Mandatory independent review (§11.4.141)

Every translation produced by the §11.4.140 pipeline **MUST** be reviewed by an **independent** model (different provider/model than the translator), acting as a native reviewer of the target language, before it ships. Tooling:

- `scripts/translation/review_translation.py` — independent reviewer engine; emits strict JSON
`{verdict, accuracy, fluency, completeness, script_ok, untranslated_leftovers, issues}`
- `scripts/translation/review-translations.sh <site-root> <lang...>`
— per-language review driver; writes per-file evidence + an aggregate `REVIEW-REPORT.md` .

Reviewer provider/model is chosen via `REVIEW_PROVIDER` / `REVIEW_MODEL` and **MUST** differ from the translator. A FAIL/ERROR blocks the content; only a real PASS ships.